



VIT[®]
AP



WEB TECHNOLOGIES

CSE-4004

MODULE 6: REST APIs & CASE STUDY



Express



By: Dr.MD.Sirajuddin

SCOPE

REST APIs

In Express.js, REST API means creating URLs (routes) that respond to HTTP methods like GET, POST, PUT, DELETE.

```
const express = require('express');
```

```
const app = express();
```

```
app.use(express.json()); //Parse JSON data & convert to JS Object
```

1. GET: Read Data

```
app.get('/products', (req, res) => {  
  res.json([  
    { id: 1, name: "Laptop" },  
    { id: 2, name: "Phone" }  
  ]);  
});
```

REST APIs

2. POST: Create Data

```
app.post('/products', (req, res) => {  
  const product = req.body;  
  res.send("Product added");  
});
```

3. PUT: Update Data

```
app.put('/products/:id', (req, res) => {  
  res.send(`Product ${req.params.id} updated`);  
});
```

:id → route parameter

Used to update existing record

REST APIs

4. DELETE: Remove Data

```
app.delete('/products/:id', (req, res) => {  
  res.send(`Product ${req.params.id} deleted` );  
});
```

Route Parameter

req.params.id

CASE STUDY: 1: E-Commerce Application

An online shopping application stores customer data in a MongoDB collection called Customers in the shoppingDB database.

```
{  
  "userId": 1,  
  "name": "Alice",  
  "email": "alice@example.com",  
  "orders": [  
    { "product": "Laptop", "price": 70000 },  
    { "product": "Mouse", "price": 500 }  
  ]  
}
```

Write a Node.js function that:

1. Connects to MongoDB Database and Collection
2. Accepts userId as input
3. Fetches the customer details

If user not found → display "Customer not found"

4. Close the connection properly

CASE STUDY: 1: E-Commerce Application

```
const { MongoClient } = require('mongodb');
const url = 'mongodb://127.0.0.1:27017';
const dbName = 'shoppingDB';
async function getCustomerById(userId) {
  const client = new MongoClient(url);
  try {
    await client.connect();
    console.log("Connected to MongoDB");
    const db = client.db(dbName);
    const collection = db.collection('Customers');
    const customer = await collection.findOne({ userId: userId });
    if (!customer) {
      console.log("Customer not found");
    } else {
      console.log("Customer details:", customer);
    } } catch (err) {
    console.error("Error:", err);
  }
  finally { await client.close();
    console.log("Connection closed"); } }
```

CASE STUDY:2: Product Management System

A company is developing a product management system where users can view, add, update, and delete products. Each product has: id, name & price. The backend is implemented using Express.js.

Write Express.js code using REST APIs to perform the following

1. Fetch all products
2. Fetch one product only
3. Add a new product
4. Update product
5. Delete product

CASE STUDY:2

1. Fetch all products

```
const express = require('express');
const app = express();
let products = [{ id: 1, name: "Laptop", price: 50000 },
  { id: 2, name: "Mobile", price: 20000 } ];
app.get('/products', (req, res) => {res.json(products); });
app.listen(3000);
```

2. Fetch one product only

```
app.get('/products/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const product = products.find(p => p.id === id);
  if (!product) {
    return res.status(404).json({ message: "Product not found" });
  }
  res.json(product); });
```


CASE STUDY:2

3. Add Product

```
app.post('/products', (req, res) => {  
  const newProduct = {  
    id: products.length + 1,  
    name: req.body.name,  
    price: req.body.price  
  };  
  products.push(newProduct);  
  res.status(201).json(newProduct);  
});
```

CASE STUDY:2

4. Update Product

```
app.put('/products/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const product = products.find(p => p.id === id);  
  if (!product) {  
    return res.status(404).json({ message: "Product not found" });  
  }  
  product.name = req.body.name;  
  product.price = req.body.price;  
  res.json(product);  
});
```

CASE STUDY:2

5. Delete Product

```
app.delete('/products/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const index = products.findIndex(p => p.id === id);  
  
  if (index === -1) {  
    return res.status(404).json({ message: "Product not found" });  
  }  
  
  products.splice(index, 1);  
  res.json({ message: "Product deleted successfully" });  
});
```